

1. Synchronisation Using Mutexes

1. Write a C program where multiple threads increment a shared counter using a mutex to prevent race conditions.

Code

```
#include <stdio.h>
#include <pthread.h>
int counter = 0;
pthread_mutex_t lock;
void* increment(void* arg)
{
    for(int i=0;i<100000;i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
}
int main()
{
    pthread_t t1,t2;
    pthread_mutex_init(&lock,NULL);
    pthread_create(&t1,NULL,increment,NULL);
    pthread_create(&t2,NULL,increment,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    printf("Final Counter = %d\n",counter);
    pthread_mutex_destroy(&lock);
}
```

Output



```
monishkanna@DESKTOP-H7HK4MK:~$ nano counter_mutex.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc counter_mutex.c -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./a.out
Final Counter = 200000
```

2. Implement a producer-consumer problem using a mutex to synchronize access to a shared buffer.

Code

```
#include <stdio.h>
#include <pthread.h>
int buffer=0;
pthread_mutex_t lock;
void* producer(void* arg)
{
    pthread_mutex_lock(&lock);
    buffer=100;
    printf("Producer produced %d\n",buffer);
    pthread_mutex_unlock(&lock);
}
void* consumer(void* arg)
{
    pthread_mutex_lock(&lock);
    printf("Consumer consumed %d\n",buffer);
    pthread_mutex_unlock(&lock);
}
```

```

}
int main()
{
    pthread_t p,c;
    pthread_mutex_init(&lock,NULL);
    pthread_create(&p,NULL,producer,NULL);
    pthread_create(&c,NULL,consumer,NULL);
    pthread_join(p,NULL);
    pthread_join(c,NULL);
    pthread_mutex_destroy(&lock);
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano prod_cons_mutex.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc prod_cons_mutex.c -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./a.out
Producer produced 100
Consumer consumed 100

```

3. Create a program where two threads alternately print numbers from 1 to 20, using a mutex to control access to the print operation.

Code

```

#include<stdio.h>
#include<pthread.h>
pthread_mutex_t lock;
void* print1(void* arg)
{
    for(int i=1;i<=20;i+=2)
    {
        pthread_mutex_lock(&lock);
        printf("%d\n",i);
        pthread_mutex_unlock(&lock);
    }
}
void* print2(void* arg)
{
    for(int i=2;i<=20;i+=2)
    {
        pthread_mutex_lock(&lock);
        printf("%d\n",i);
        pthread_mutex_unlock(&lock);
    }
}
int main()
{
    pthread_t t1,t2;
    pthread_mutex_init(&lock,NULL);
    pthread_create(&t1,NULL,print1,NULL);
    pthread_create(&t2,NULL,print2,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    pthread_mutex_destroy(&lock);
}

```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano alternate_mutex.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc alternate_mutex.c -pthread
./a.out
2
4
6
8
10
12
14
16
18
20
1
3
5
7
9
11
13
15
17
19
```

2. Synchronisation Using Conditional Variables

1. Write a C program that uses a condition variable to signal a consumer thread when a producer thread adds data to a shared buffer.

Code

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define SIZE 5
int buffer[SIZE];
int count = 0;
pthread_mutex_t mutex;
pthread_cond_t cond;
void* producer(void* arg)
{
    for(int i=1;i<=5;i++)
    {
        pthread_mutex_lock(&mutex);
        buffer[count++] = i;
        printf("Produced: %d\n", i);
        pthread_cond_signal(&cond); // notify consumer
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}
void* consumer(void* arg)
```

```

{
    for(int i=1;i<=5;i++)
    {
        pthread_mutex_lock(&mutex);
        while(count == 0)
            pthread_cond_wait(&cond, &mutex);
        int item = buffer[--count];
        printf("Consumed: %d\n", item);
        pthread_mutex_unlock(&mutex);
    }
}

int main()
{
    pthread_t p, c;
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano prod_cons_cond.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc prod_cons_cond.c -pthread
./a.out
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

```

2. Implement a thread pool where worker threads wait on a condition variable for new tasks.

Code

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define THREADS 3
#define TASKS 5
int task_count = 0;
pthread_mutex_t mutex;
pthread_cond_t cond;
void* worker(void* arg)
{
    while(1)
    {
        pthread_mutex_lock(&mutex);

```

```

        while(task_count == 0)
            pthread_cond_wait(&cond,&mutex);
        task_count--;
        printf("Thread %ld processing task\n", pthread_self());
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}
int main()
{
    pthread_t workers[THREADS];
    pthread_mutex_init(&mutex,NULL);
    pthread_cond_init(&cond,NULL);
    for(int i=0;i<THREADS;i++)
        pthread_create(&workers[i],NULL,worker,NULL);
    for(int i=0;i<TASKS;i++)
    {
        sleep(1);
        pthread_mutex_lock(&mutex);
        task_count++;
        printf("Task added\n");
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
    }
    sleep(5);
    return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano thread_pool.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc thread_pool.c -pthread
./a.out
Task added
Thread 134958748268224 processing task
Task added
Thread 134958739875520 processing task
Task added
Thread 134958731482816 processing task
Task added
Thread 134958748268224 processing task
Task added
Thread 134958739875520 processing task

```

3. Create a program where one thread waits for a specific condition (e.g., counter reaches 10) before proceeding, using a condition variable for synchronization.

Code

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
int counter = 0;
pthread_mutex_t mutex;
pthread_cond_t cond;

```

```

void* increment(void* arg)
{
    for(int i=0;i<10;i++)
    {
        sleep(1);
        pthread_mutex_lock(&mutex);
        counter++;
        printf("Counter: %d\n",counter);
        if(counter == 10)
            pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
    }
}

void* waiter(void* arg)
{
    pthread_mutex_lock(&mutex);
    while(counter < 10)
        pthread_cond_wait(&cond,&mutex);
    printf("Counter reached 10! Thread proceeding...\n");
    pthread_mutex_unlock(&mutex);
}

int main()
{
    pthread_t t1,t2;
    pthread_mutex_init(&mutex,NULL);
    pthread_cond_init(&cond,NULL);
    pthread_create(&t1,NULL,increment,NULL);
    pthread_create(&t2,NULL,waiter,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano cond_counter.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc cond_counter.c -pthread
./a.out
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
Counter: 6
Counter: 7
Counter: 8
Counter: 9
Counter: 10
Counter reached 10! Thread proceeding...

```

3. IPC - Unnamed Pipes

1. Write a C program where a parent process sends a string to its child process using an unnamed pipe.

Code

```
#include <stdio.h>
```

```

#include <unistd.h>
#include <string.h>
int main()
{
    int fd[2];
    pid_t pid;
    char msg[] = "Hello from Parent!";
    char buffer[100];
    pipe(fd); // create pipe
    pid = fork();
    if(pid > 0) // Parent
    {
        close(fd[0]); // close read end
        write(fd[1], msg, strlen(msg)+1);
        close(fd[1]);
    }
    else // Child
    {
        close(fd[1]); // close write end
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd[0]);
    }
    return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano pipe_parent_child.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc pipe_parent_child.c
./a.out
Child received: Hello from Parent!

```

2. Implement a program where two child processes communicate through an unnamed pipe created by their parent.

Code

```

#include <stdio.h>
#include <unistd.h>
#include <string.h>
int main()
{
    int fd[2];
    pipe(fd);
    if(fork() == 0) // Child 1 (Writer)
    {
        char msg[] = "Message from Child 1";
        close(fd[0]);
        write(fd[1], msg, strlen(msg)+1);
        close(fd[1]);
    }
    else
    {
        if(fork() == 0) // Child 2 (Reader)
        {
            char buffer[100];

```

```

        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Child 2 received: %s\n", buffer);
        close(fd[0]);
    }
}
return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano pipe_two_children.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc pipe_two_children.c
./a.out
Child 2 received: Message from Child 1

```

3. Create a program that uses an unnamed pipe to transfer the contents of a file from one process to another.

Code

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid;
    char buffer[100];
    pipe(fd);
    pid = fork();
    if(pid > 0) // Parent
    {
        int file = open("input.txt", O_RDONLY);
        if(file < 0)
        {
            perror("File open failed");
            return 1;
        }
        close(fd[0]);
        int n;
        while((n = read(file, buffer, sizeof(buffer))) > 0)
        {
            write(fd[1], buffer, n);
        }
        close(file);
        close(fd[1]);
        wait(NULL); // wait for child
    }
    else // Child
    {
        close(fd[1]);
        int n;
        while((n = read(fd[0], buffer, sizeof(buffer))) > 0)
        {
            write(1, buffer, n); // print to terminal
        }
    }
}

```



```
    close(fd[0]);  
}  
return 0;
```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano pipe_file.c  
gcc pipe_file.c  
monishkanna@DESKTOP-H7HK4MK:~$ echo "Hello Pipe IPC Working!" > input.txt  
monishkanna@DESKTOP-H7HK4MK:~$ ./a.out  
Hello Pipe IPC Working!
```

4. IPC - Named Pipes

1. Write a C program that creates a named pipe (FIFO) and uses it for communication between two unrelated processes.

Writer

Code

```
#include <stdio.h>  
#include <fcntl.h>  
#include <sys/stat.h>  
#include <unistd.h>  
int main()  
{  
    char *fifo = "myfifo";  
    mkfifo(fifo, 0666); // create FIFO  
    int fd = open(fifo, O_WRONLY);  
    char msg[] = "Hello from Writer Process!";  
    write(fd, msg, sizeof(msg));  
    close(fd);  
    return 0;  
}
```

Reader

```
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
int main()  
{  
    char *fifo = "myfifo";  
    char buffer[100];  
    int fd = open(fifo, O_RDONLY);  
    read(fd, buffer, sizeof(buffer));  
    printf("Received: %s\n", buffer);  
    close(fd);  
    return 0;  
}
```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano fifo_writer.c  
monishkanna@DESKTOP-H7HK4MK:~$ gcc fifo_writer.c  
./a.out
```

```
monishkanna@DESKTOP-H7HK4MK:~$ nano fifo_reader.c
monishkanna@DESKTOP-H7HK4MK:~$ ^C
monishkanna@DESKTOP-H7HK4MK:~$ gcc fifo_reader.c
./a.out
Received: Hello from Writer Process!
```

2. Implement a client-server model using named pipes, where the client sends requests and the server responds.

Server

Code

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>
int main()
{
    char *fifo = "serverfifo";
    mkfifo(fifo, 0666);
    char buffer[100];
    int fd = open(fifo, O_RDWR);
    read(fd, buffer, sizeof(buffer));
    printf("Client says: %s\n", buffer);
    char reply[] = "Message received by server!";
    write(fd, reply, strlen(reply)+1);
    close(fd);
    return 0;
}
```

Client

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main()
{
    char *fifo = "serverfifo";
    char buffer[100];
    int fd = open(fifo, O_RDWR);
    char msg[] = "Hello Server!";
    write(fd, msg, strlen(msg)+1);
    read(fd, buffer, sizeof(buffer));
    printf("Server replied: %s\n", buffer);
    close(fd);
    return 0;
}
```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano server.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc server.c
monishkanna@DESKTOP-H7HK4MK:~$ ./a.out
Client says: Hello Server!
```

```
monishkanna@DESKTOP-H7HK4MK:~$ nano client.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc client.c
monishkanna@DESKTOP-H7HK4MK:~$ ./a.out
Server replied: Message received by server!
```

3. Create a program that writes log messages to a named pipe, which are read and displayed by a separate process.

Writer

Code

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <time.h>
int main()
{
    char *fifo = "logfifo";
    mkfifo(fifo, 0666);
    int fd = open(fifo, O_WRONLY);
    for(int i=1;i<=5;i++)
    {
        char log[100];
        sprintf(log, "Log %d: Event occurred\n", i);
        write(fd, log, sizeof(log));
        sleep(1);
    }
    close(fd);
    return 0;
}
```

Reader

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    char *fifo = "logfifo";
    char buffer[100];
    int fd = open(fifo, O_RDONLY);
    while(read(fd, buffer, sizeof(buffer)) > 0)
    {
        printf("LOG: %s", buffer);
    }
    close(fd);
    return 0;
}
```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano log_reader.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc log_reader.c
monishkanna@DESKTOP-H7HK4MK:~$ ./a.out
LOG: Log 1: Event occurred
LOG: Log 2: Event occurred
LOG: Log 3: Event occurred
LOG: Log 4: Event occurred
LOG: Log 5: Event occurred

monishkanna@DESKTOP-H7HK4MK:~$ nano log_writer.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc log_writer.c
monishkanna@DESKTOP-H7HK4MK:~$ ./a.out
```

5. IPC - Message Queues

1. Write a C program that creates a System V message queue and sends messages between two processes.

Sender

Code

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct msg_buffer {
    long msg_type;
    char msg_text[100];
};

int main()
{
    key_t key = ftok("progfile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msg_buffer message;
    message.msg_type = 1;
    strcpy(message.msg_text, "Hello from Sender!");
    msgsnd(msgid, &message, sizeof(message.msg_text), 0);
    printf("Message sent\n");
    return 0;
}
```

Receiver

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct msg_buffer {
    long msg_type;
    char msg_text[100];
};

int main()
{
    key_t key = ftok("progfile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msg_buffer message;
```

```

msgrcv(msgid, &message, sizeof(message.msg_text), 1, 0);
printf("Received: %s\n", message.msg_text);
msgctl(msgid, IPC_RMID, NULL); // delete queue
return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano sender.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sender.c
./a.out
Message sent

monishkanna@DESKTOP-H7HK4MK:~$ nano receiver.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc receiver.c
./a.out
Received: Hello from Sender!

```

2. Implement a program where multiple clients send messages to a server process using a message queue, and the server processes them in order.

Code

Server

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct msg_buffer {
    long msg_type;
    char msg_text[100];
};
int main()
{
    key_t key = ftok("progrfile", 75);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msg_buffer message;
    while(1)
    {
        msgrcv(msgid, &message, sizeof(message.msg_text), 1, 0);
        printf("Server received: %s\n", message.msg_text);
        if(strcmp(message.msg_text, "exit") == 0)
            break;
    }
    msgctl(msgid, IPC_RMID, NULL);
    return 0;
}

```

Receiver

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
struct msg_buffer {
    long msg_type;
    char msg_text[100];
};

```

```

int main()
{
    key_t key = ftok("progfile", 75);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msg_buffer message;
    message.msg_type = 1;
    printf("Enter message: ");
    fgets(message.msg_text, sizeof(message.msg_text), stdin);
    msgsnd(msgid, &message, sizeof(message.msg_text), 0);
    return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano server.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc server.c -o server
monishkanna@DESKTOP-H7HK4MK:~$ ./server
Server received: Hello
Server received: Monish kannu

```

```

monishkanna@DESKTOP-H7HK4MK:~$ nano client.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc client.c -o client
monishkanna@DESKTOP-H7HK4MK:~$ ./client
Enter message: Hello
monishkanna@DESKTOP-H7HK4MK:~$ ./client
Enter message: Monish kannu

```

3. Create a program that demonstrates message queue attributes, such as maximum message size and queue length.

Code

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int main()
{
    key_t key = ftok("progfile", 90);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msqid_ds info;
    msgctl(msgid, IPC_STAT, &info);
    printf("Maximum bytes allowed: %ld\n", info.msg_qbytes);
    printf("Current messages in queue: %ld\n", info.msg_qnum);
    printf("Last sender PID: %d\n", info.msg_lspid);
    printf("Last receiver PID: %d\n", info.msg_lrpid);
    msgctl(msgid, IPC_RMID, NULL); // remove queue
    return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano msg_attr.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc msg_attr.c -o msg_attr
monishkanna@DESKTOP-H7HK4MK:~$ ./msg_attr
Maximum bytes: 16384
Messages: 0
Sender PID: 1931
Receiver PID: 1916

```

6. Shared Memory

1. Write a C program that creates a shared memory segment and allows two processes to exchange data through it

Writer

Code

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
int main()
{
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char*) shmat(shmid, NULL, 0);

    printf("Write Data: ");
    fgets(str, 100, stdin);
    printf("Data written in memory: %s\n", str);
    shmdt(str);
    return 0;
}
```

Reader

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main()
{
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char*) shmat(shmid, NULL, 0);
    printf("Data read from memory: %s\n", str);
    shmdt(str);
    shmctl(shmid, IPC_RMID, NULL); // delete shared memory
    return 0;
}
```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano shm_write.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc shm_write.c -o write
monishkanna@DESKTOP-H7HK4MK:~$ ./write
Write Data: hello Everyone!!:)
Data written in memory: hello Everyone!!:)
```

```
Read: monishkanna@DESKTOP-H7HK4MK:~$ nano shm_read.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc shm_read.c -o read
monishkanna@DESKTOP-H7HK4MK:~$ ./read
Data read from memory: hello Everyone!!:)
```

2. Implement a producer-consumer problem using shared memory for the buffer and semaphores for synchronization.

Code

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <unistd.h>
#define SIZE 5
int main()
{
    key_t key = ftok("shmfile", 75);
    int shmid = shmget(key, SIZE * sizeof(int), 0666 | IPC_CREAT);
    int *buffer = (int*) shmat(shmid, NULL, 0);
    int semid = semget(key, 2, 0666 | IPC_CREAT);
    // sem[0] = empty, sem[1] = full
    semctl(semid, 0, SETVAL, SIZE);
    semctl(semid, 1, SETVAL, 0);
    struct sembuf wait_empty = {0, -1, 0};
    struct sembuf signal_full = {1, +1, 0};
    if(fork() == 0) // Consumer
    {
        struct sembuf wait_full = {1, -1, 0};
        struct sembuf signal_empty = {0, +1, 0};
        for(int i=0;i<5;i++)
        {
            semop(semid, &wait_full, 1);
            printf("Consumed: %d\n", buffer[i]);
            semop(semid, &signal_empty, 1);
        }
    }
    else // Producer
    {
        for(int i=0;i<5;i++)
        {
            semop(semid, &wait_empty, 1);
            buffer[i] = i+1;
            printf("Produced: %d\n", buffer[i]);
            semop(semid, &signal_full, 1);
            sleep(1);
        }
    }
    return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano pc.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc pc.c -o pc
monishkanna@DESKTOP-H7HK4MK:~$ ./pc
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

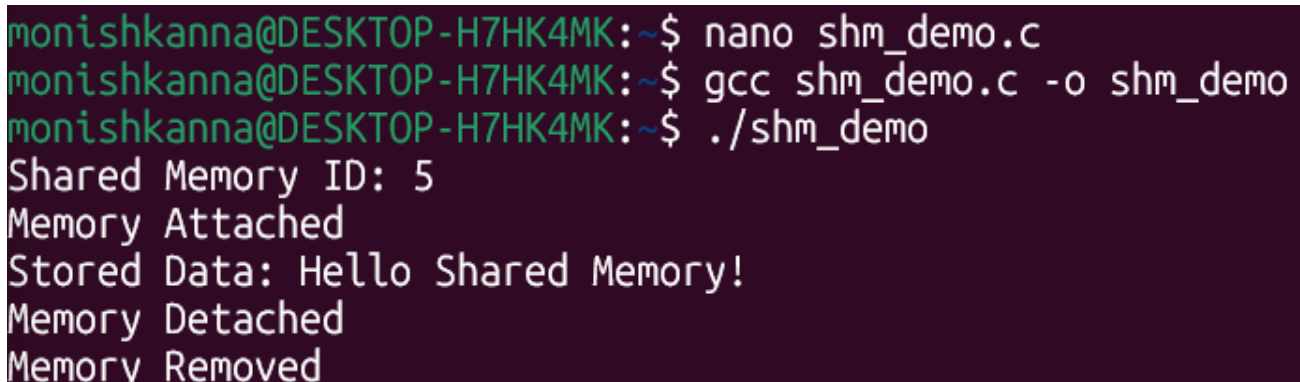
```


3. Create a program that demonstrates how to attach, detach, and remove a shared memory segment.

Code

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int main()
{
    key_t key = ftok("shmfile", 90);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    printf("Shared Memory ID: %d\n", shmid);
    char *data = (char*) shmat(shmid, NULL, 0);
    printf("Memory Attached\n");
    sprintf(data, "Hello Shared Memory!");
    printf("Stored Data: %s\n", data);
    shmdt(data);
    printf("Memory Detached\n");
    shmctl(shmid, IPC_RMID, NULL);
    printf("Memory Removed\n");
    return 0;
}
```

Output



```
monishkanna@DESKTOP-H7HK4MK:~$ nano shm_demo.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc shm_demo.c -o shm_demo
monishkanna@DESKTOP-H7HK4MK:~$ ./shm_demo
Shared Memory ID: 5
Memory Attached
Stored Data: Hello Shared Memory!
Memory Detached
Memory Removed
```

7. Semaphores

1. Write a C program that uses POSIX semaphores to synchronize access to a shared resource among multiple processes.

Code

```
#include <stdio.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    sem_t *sem = sem_open("/mysem", O_CREAT, 0666, 1); // binary semaphore

    for(int i = 0; i < 2; i++)
    {
        if(fork() == 0)
        {
            sem_wait(sem); // enter critical section
```

```

        printf("Process %d entering critical section\n", getpid());
        sleep(2);
        printf("Process %d leaving critical section\n", getpid());
        sem_post(sem); // exit
        return 0;
    }
}
wait(NULL);
wait(NULL);
sem_close(sem);
sem_unlink("/mysem");
return 0;
}

```

Output

```

monishkanna@DESKTOP-H7HK4MK:~$ nano sem_process.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sem_process.c -o sem_process -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./sem_process
Process 2191 entering critical section
Process 2191 leaving critical section
Process 2192 entering critical section
Process 2192 leaving critical section

```

2. Implement a program where a semaphore controls access to a critical section, allowing only three processes at a time.

Code

```

#include <stdio.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    sem_t *sem = sem_open("/limitsem", O_CREAT, 0666, 3); // allow 3
    for(int i = 0; i < 6; i++)
    {
        if(fork() == 0)
        {
            sem_wait(sem);
            printf("Process %d in critical section\n", getpid());
            sleep(3);
            printf("Process %d leaving\n", getpid());
            sem_post(sem);
            return 0;
        }
    }
    for(int i=0;i<6;i++)
        wait(NULL);
    sem_close(sem);
    sem_unlink("/limitsem");
    return 0;
}

```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano sem_limit.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sem_limit.c -o sem_limit -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./sem_limit
Process 2206 in critical section
Process 2207 in critical section
Process 2208 in critical section
Process 2207 leaving
Process 2209 in critical section
Process 2206 leaving
Process 2210 in critical section
Process 2208 leaving
Process 2211 in critical section
Process 2210 leaving
Process 2209 leaving
Process 2211 leaving
```

3. Create a program that demonstrates the use of semaphore operations (sem_wait, sem_post) in a multi-threaded environment.

Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
sem_t sem;
void* task(void* arg)
{
    sem_wait(&sem); // lock
    printf("Thread %ld entering\n", pthread_self());
    sleep(2);
    printf("Thread %ld leaving\n", pthread_self());
    sem_post(&sem); // unlock
}
int main()
{
    pthread_t t1, t2, t3;
    sem_init(&sem, 0, 1); // binary semaphore
    pthread_create(&t1, NULL, task, NULL);
    pthread_create(&t2, NULL, task, NULL);
    pthread_create(&t3, NULL, task, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    sem_destroy(&sem);
    return 0;
}
```

Output

```
monishkanna@DESKTOP-H7HK4MK:~$ nano sem_thread.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sem_thread.c -o sem_thread -pthread
monishkanna@DESKTOP-H7HK4MK:~$ ./sem_thread
Thread 124984739493568 entering
Thread 124984739493568 leaving
Thread 124984722708160 entering
Thread 124984722708160 leaving
Thread 124984731100864 entering
Thread 124984731100864 leaving
```